CLÍNICA VIVA

Guia para a apresentação

Roteiro de demonstração, arquitetura e respostas comentadas para estudar o funcionamento do sistema.

Análise e Projeto de Sistemas II
26 de outubro de 2026

Protótipo acadêmico com dados fictícios.
Nomes completos e RGMs do grupo: preencher com os integrantes reais.

Sistema: https://clinica-viva-aps.vercel.app
Código: https://github.com/Ygxxrr/clinica-viva-aps

Guia de apresentação - Clínica Viva

Explicação inicial, em suas palavras

"O projeto centraliza a gestão de uma clínica médica. A ideia é acompanhar a jornada do paciente: cadastro, agendamento, atendimento, exames, medicamentos, procedimentos e financeiro. O backend usa Python com Flask, e o banco relacional organiza os vínculos entre essas informações. A interface usa HTML, CSS, JavaScript e templates Jinja."

Use este texto como apoio de estudo, sem decorar uma autoria ou um histórico que não correspondam ao trabalho real do grupo. Demonstre apenas o que você consegue reproduzir e explicar.

Demonstração sugerida - aproximadamente 8 a 10 minutos

1. **Login e visão geral - 45 segundos.** Entre com o administrador. Explique que existem quatro perfis e que os números são calculados a partir do banco, com dados fictícios.

2. **Pacientes e prontuário - 1 minuto.** Abra Ana Clara Martins. Mostre CPF, convênio, alergia, prontuário único e a linha do tempo. Explique que o histórico pode conter diferentes médicos.

3. **Estrutura da clínica - 45 segundos.** Mostre os cadastros de profissionais e unidades. Explique as relações muitos para muitos com especialidades e unidades.

4. **Agenda - 1 minuto.** Filtre hoje e abra um agendamento ativo. Se quiser criar outro, use um horário de 30 minutos livre. Demonstre que o sistema valida médico, especialidade, unidade e conflito de horário.

5. **Atendimento - 1 minuto.** Abra um agendamento da Dra. Helena e preencha queixa, diagnóstico e conduta com um caso explicitamente fictício. Salve e mostre o agendamento Realizado. Não use um horário que já tenha atendido durante o ensaio.

6. **Exames, prescrição e procedimento - 2 minutos.** Solicite um exame; crie uma prescrição com dois medicamentos fictícios; registre um procedimento com valor diferente do padrão. Mostre como tudo continua vinculado à consulta.

7. **Financeiro - 1 minuto.** Crie uma cobrança para a consulta. Por exemplo: total R$ 200, cobertura R$ 120 para o convênio ativo de Ana, paciente R$ 80. Registre R$ 80 por PIX e R$ 120 por Convênio; o saldo deve zerar. O valor é didático, não uma tarifa real.

8. **Relatórios - 45 segundos.** Filtre o período. Mostre faturamento por médico/especialidade/unidade e quantidade de consultas. Explique a diferença entre faturamento emitido e recebimento.

9. **Arquitetura e testes - 1 minuto.** Mostre a árvore de arquivos, os diagramas e o comando de testes. Explique uma regra concreta em services.py.

Se o tempo for curto, use os dados pré-carregados e apresente cadastro → consulta → financeiro → relatório. Faça um ensaio antes para conhecer os nomes e os horários livres.

Perguntas prováveis do professor

Qual linguagem foi utilizada? Por quê?

Python no servidor, usando Flask. A sintaxe é legível e o framework permite escrever rotas HTTP explícitas, facilitando conectar cada tela à sua funcionalidade. No navegador há HTML para estrutura, CSS para aparência e JavaScript para pequenas interações. SQL é a linguagem utilizada pelo banco, gerada principalmente pelo SQLAlchemy.

Qual é a diferença entre Python e Flask?

Python é a linguagem. Flask é uma biblioteca/framework web escrita em Python que recebe requisições HTTP, identifica a rota e devolve a resposta.

O que é uma rota?

É a associação entre uma URL, um método HTTP e uma função Python. Por exemplo, GET /agenda mostra os agendamentos; POST /agenda/novo valida o formulário e grava um agendamento.

Como a tela se conecta ao banco?

O formulário envia um POST ao Flask. A rota verifica sessão, perfil e token CSRF. Uma função de services.py valida as regras. Os objetos definidos em models.py são manipulados pelo SQLAlchemy, que envia comandos parametrizados ao banco. O commit confirma a transação. Depois a rota redireciona para uma página atualizada.

Por que usar POST e depois redirecionar?

Para que atualizar a página de resultado faça uma nova leitura, sem reenviar o mesmo formulário. É o padrão Post/Redirect/Get. Restrições no banco também impedem duplicação de uma consulta para o mesmo agendamento.

O que é ORM? Por que SQLAlchemy?

Mapeamento objeto-relacional: a classe Patient corresponde à tabela patient; uma instância corresponde a uma linha. SQLAlchemy auxilia consultas, relacionamentos e transações, reduz SQL repetitivo e permite usar SQLite local e PostgreSQL hospedado. Não elimina a necessidade de entender chaves, tabelas e joins.

Qual banco é utilizado?

Localmente, SQLite: guarda os dados em um arquivo e facilita executar sem instalar um servidor. Na hospedagem Vercel é necessário PostgreSQL externo para persistência, porque o armazenamento local das funções não é uma base permanente. Consulte a configuração realmente publicada antes de afirmar que o deploy está funcionando.

Qual é a função do GitHub e da Vercel?

GitHub armazena e versiona o código. Vercel hospeda e executa a aplicação web. O banco PostgreSQL é um terceiro componente, responsável pelos dados persistentes. Uma URL de repositório não é a mesma coisa que a URL do sistema.

Por que separar agendamento de consulta?

Agendamento representa a intenção de atendimento; pode ser confirmado, cancelado ou resultar em falta. Consulta representa o atendimento realizado, com diagnóstico e conduta. Misturar os dois permitiria tratar uma marcação cancelada como atendimento efetivo.

Como um cancelamento é validado?

services.change_appointment define as transições permitidas. services.attend só aceita Agendado ou Confirmado. Na mesma transação cria Consultation e muda a situação para Realizado. A coluna appointment_id da consulta é única, impedindo dois atendimentos para a mesma marcação.

Como funciona o conflito de horários?

A aplicação usa slots de 30 minutos. Antes de salvar, procura outro atendimento ativo do mesmo médico ou do mesmo paciente naquele intervalo. Cancelados e faltantes não bloqueiam o horário. Também valida se o médico atende a especialidade e a unidade escolhidas.

Existe proteção total contra dois agendamentos simultâneos?

Não. A verificação atual é feita na camada de serviço. Em concorrência alta, dois pedidos poderiam passar pela consulta de conflito ao mesmo tempo. Para uma operação real, adicionaríamos uma restrição/índice apropriado ou exclusão de intervalos no PostgreSQL e testes concorrentes. O bloqueio de linha existente protege operações sobre agendamentos e cobranças já existentes; não transforma a verificação de slots em uma garantia de unicidade concorrente.

Por que paciente e convênio precisam de outra tabela?

Porque é muitos para muitos e o vínculo tem atributos próprios. A carteirinha é do paciente naquele convênio, não do cadastro geral do paciente ou da operadora. PatientInsurance guarda paciente, convênio, carteirinha, adesão e situação.

Onde ficam os dados do paciente em uma solicitação de exame?

O pedido referencia a consulta. A consulta referencia o agendamento, que identifica o paciente. A propriedade patient percorre esse vínculo. Não duplicamos um patient_id no pedido para evitar que um exame aponte para um paciente e sua consulta para outro.

Qual é a diferença entre medicamento e item da prescrição?

Medication é o catálogo reutilizável com princípio ativo, fabricante e apresentação. PrescriptionItem é o uso específico naquele tratamento, com quantidade, dose, frequência, duração e observações. O mesmo medicamento pode ser prescrito de formas diferentes em consultas diferentes.

Por que guardar o preço do procedimento realizado?

O preço padrão é uma referência que pode mudar. O valor do atendimento passado precisa permanecer como foi praticado. Por isso Procedure.standard_price é separado de PerformedProcedure.charged_price.

Qual a diferença entre cobrança e pagamento?

Charge representa o que é devido. Payment representa um registro de pagamento, com data, valor, forma e situação. Uma cobrança pode ser quitada em várias parcelas e por responsáveis diferentes. Apenas registros Pago reduzem o saldo e entram nos recebimentos.

Como o convênio é contabilizado?

Na cobrança, total menos cobertura resulta na parte do paciente. Pagamento com forma Convênio reduz o saldo do convênio; as demais formas reduzem o saldo do paciente. Nenhuma parte pode pagar acima do seu saldo. A aplicação não transfere dinheiro e não se comunica com a operadora: registra as informações internamente.

Por que Decimal em vez de float?

Float representa números em base binária e pode produzir diferenças de arredondamento. Decimal e Numeric permitem trabalhar com valores monetários em duas casas decimais.

O relatório soma cobranças ou pagamentos?

Mostra os dois separadamente. Faturamento emitido soma cobranças criadas no período. Recebimentos somam pagamentos Pago com data dentro do período. O médico, a especialidade e a unidade vêm da consulta de origem. Quantidade de consultas conta atendimentos registrados, independentemente de haver pagamento.

Como funciona a autenticação?

A senha é comparada com um hash gerado pelo Werkzeug. A sessão assinada identifica o usuário, e a autorização é validada no servidor em cada rota. A interface esconde ações sem permissão, mas esconder botões não é a proteção principal. Cookies usam HttpOnly e SameSite; na Vercel, também Secure.

O que é CSRF?

Um ataque em que outra página tenta submeter ações usando a sessão do usuário. Cada formulário POST envia um token aleatório associado à sessão. O servidor rejeita token ausente ou diferente. Isso complementa o controle de sessão.

Como evita SQL injection e scripts no nome do paciente?

As consultas utilizam parâmetros do SQLAlchemy, sem concatenar o texto digitado em comandos SQL. Na saída, Jinja escapa o HTML. Um nome contendo <script> deve aparecer como texto, sem executar. Existe teste disso.

Quais testes foram feitos?

Uma suíte de integração com banco em memória valida todas as telas, controle de acesso, CSRF, CPF inválido/duplicado, criação automática de prontuário, conflito de agenda, bloqueio de cancelados e o fluxo completo com exame, prescrição de dois itens, procedimento e pagamentos. A existência dessa suíte não significa teste de carga, auditoria de segurança ou certificação para uso real.

Qual padrão de arquitetura foi utilizado?

Uma aplicação web em camadas, com inspiração em MVC: modelos em models.py; templates como apresentação; rotas como controladores; services.py concentra regras de negócio. É um monólito modular, adequado ao tamanho do protótipo. Não há microserviços nem uma API separada de frontend.

Por que não criar um frontend separado?

Renderização no servidor reduz ferramentas e comunicação entre dois projetos. A página já chega com os dados necessários. JavaScript fica responsável por modal, adição de itens e seleção de opções, sem duplicar as regras definitivas.

O que você melhoraria depois?

Migrações versionadas, testes de concorrência e carga, restrições concorrentes para agenda, controle de autoria/versionamento clínico, gestão de usuários, recuperação de senha, cópias de segurança, estorno financeiro, paginação, restrição mais fina de acesso a prontuários e integrações de laboratório/convênio. O modo de demonstração precisa ser removido antes de qualquer uso real.

Onde estudar no código

1. models.py: PatientInsurance, Appointment, Consultation, PrescriptionItem, Charge e Payment.
2. services.py: schedule, attend, prescribe, create_charge e pay.
3. app.py: login, roles, authenticate, consultation_new e payment_new.
4. templates/consultation.html: como um formulário aponta para uma rota.
5. tests/test_system.py: o teste test_end_to_end_consultation_exams_prescription_procedure_payment.

Checklist antes de sair

- Abrir o link publicado em uma janela sem sessão existente e testar login.
- Conferir se os dados persistem após atualizar a página.
- Testar uma consulta e um pagamento de demonstração.
- Guardar uma cópia local do projeto e saber iniciar com Python se faltar internet.
- Confirmar nomes completos e RGMs reais do grupo e as instruções de entrega do professor.
- Revisar o que foi realmente implementado e explicar as limitações sem prometer recursos ausentes.